

Buenos Aires
aprende

Agencia de Habilidades
para el Futuro

DOCUMENTACIÓN - Parte 1

Desarrollo de Sistemas Web (Back End) — 2° cuatrimestre 2026

Caso 2: Distribución de Pedidos — “FreshRoute” (FreshRoute B2B — logística de distribución refrigerada)

Empresa de Desarrollo: LosBuleanos

Integrantes:

Nicolás Zalazar
Laura Olivera
Christian Albornoz
Laura Belén Blanco
Fernando Guevara

Nombre del documento / carpeta Drive: DSWB_2A_grupo17-losbuleanos_2C26

ÍNDICE

1. Links	3
2. Bibliografía	3
3. Introducción	3
4. Integrantes y responsabilidades	4
5. Descripción funcional de la aplicación	4
5.1 Objetivo general	4
5.2 Funcionalidades principales actuales	4
6. Módulos del proyecto	5
7. Persistencia de Datos	6

1. Links

Recurso	Link
Carpeta Drive (Primera Entrega)	https://drive.google.com/drive/folders/1ibVRI3154gZ3MLaRu_Ih6XabXx1RM2T?usp=drive_link
Repositorio GitHub	https://github.com/Nicoalazar/PrimerParcialDSWB
Video de presentación	https://drive.google.com/file/d/1phkYsp872mwYkyRmzw7msNFFvt_oeUnf/view?usp=drive_link

2. Bibliografía

Material de la cátedra

- Código de clases 1 a 4 (Drive del profesor) — <https://drive.google.com/drive/folders/18hYfMEMIDmce3HxpmjJRqGdp9KBO9t9j>

Documentación

- Express 5: routing, middleware, manejo de errores y guía de migración — <https://expressjs.com/>
- Node.js: módulos fs y path — <https://nodejs.org/api/fs.html> · <https://nodejs.org/api/path.html>
- Pug: referencia del lenguaje e integración con Express — <https://pugjs.org/>
- Postman: importar/exportar colecciones — <https://learning.postman.com/docs/getting-started/importing-and-exporting/importing-and-exporting-overview/>
- MDN: métodos HTTP y códigos de estado — <https://developer.mozilla.org/es/docs/Web/HTTP/Status>

El detalle completo está en `docs/bibliografia.md` del repositorio.

3. Introducción

En esta primera instancia desarrollamos el backend de **FreshRoute B2B**, una empresa ficticia de logística de distribución refrigerada que reparte insumos a restaurantes, comedores escolares y cocinas industriales.

La aplicación está construida con **Node.js y Express 5** siguiendo el esquema **MVC**: las rutas (`routes/`) reciben el request, los controladores (`controllers/`) resuelven la lógica de cada endpoint, los modelos (`models/`) representan las entidades del dominio como clases, y las vistas (`views/`) renderizan la interfaz con **Pug**. La persistencia se resuelve con **archivos JSON** dentro de `/data`, leídos y escritos con el módulo `fs` a través de un repositorio genérico (`JsonRepository`).

Se implementaron **dos módulos funcionales con CRUD completo — Clientes y Pedidos —**, cada uno accesible por una API REST (probada con Postman) y por vistas web. Además se agregaron middlewares

de logging, validación de datos y manejo centralizado de errores, y un servicio de negocio (`PedidoService`) que valida referencias entre entidades y controla las transiciones de estado de un pedido. Los choferes se cargan desde un seed de solo lectura y se usan como referencia al crear pedidos; su CRUD y la migración a MongoDB quedan para la segunda entrega.

4. Integrantes y responsabilidades

Integrante	Responsabilidad	Sprints
Nicolás Zalazar	Estructura base del proyecto: <code>app.js</code> , configuración de Express y Pug, estructura MVC de carpetas y seed de choferes	1
Laura Olivera	<code>JsonRepository</code> genérico, Backend Clientes (model, controller, router) y vistas web de Clientes	2, 6
Christian Albornoz	Backend Pedidos (model, controller, router) + seed de choferes expuesto en <code>GET /api/choferes</code> , y vistas web de Pedidos	3, 7
Laura Belén Blanco	<code>PedidoService</code> (validación de cliente/chofer, transiciones de estado) y middlewares: logger, validación de body y manejo centralizado de errores / 404	4, 5
Fernando Guevara	QA: code review cruzado, bug bash end-to-end, smoke test automatizado y corrección de bugs · documentación y evidencia	8, 9

El trabajo se organizó en 10 sprints cortos (2 por día) con un GitHub Project, una rama por sprint y pull requests con revisión cruzada antes de integrar a `development` y luego a `main` .

5. Descripción funcional de la aplicación

5.1 Objetivo general

Centralizar la gestión de los pedidos de distribución refrigerada de FreshRoute: registrar los clientes a los que se reparte, cargar los pedidos con sus ítems, asignarlos a un chofer y seguir el avance de cada entrega a través de sus estados, garantizando que los datos sean consistentes (no se puede crear un pedido para un cliente o chofer inexistente ni saltar etapas de la entrega).

5.2 Funcionalidades principales actuales

- **ABM de clientes:** alta, listado, consulta por id, edición parcial y baja. Cada cliente tiene nombre, dirección, zona, contacto y horario de entrega.
- **ABM de pedidos:** alta, listado, consulta por id, edición parcial y baja. Cada pedido referencia a un cliente y a un chofer, tiene una lista de ítems (descripción + cantidad), una fecha/hora programada y un estado.
- **Ciclo de vida del pedido:** todo pedido nace en `pendiente` y solo puede avanzar de a un paso: `pendiente` → `asignado` → `en tránsito` → `entregado` . Las transiciones inválidas (saltos o retrocesos) se rechazan con `400` .

- **Validación de datos:** los `POST / PUT` de ambos módulos pasan por middlewares que verifican campos obligatorios y tipos, y responden `400 { mensaje, errores }` con el detalle de cada problema.
- **Integridad referencial:** al crear o editar un pedido se comprueba que el `clienteId` exista en `clientes.json` y el `choferId` en `choferes.json`.
- **Manejo de errores centralizado:** id inexistente → `404`, ruta no montada → `404 { mensaje: "Ruta no encontrada" }`, errores no previstos → `500`. Los controladores no llevan `try/catch`: Express 5 deriva los errores al `errorHandler`.
- **Logger:** cada request se registra en consola con timestamp, método y URL.
- **Interfaz web:** vistas Pug para navegar y operar ambos módulos desde el navegador (listados, formularios de alta/edición, detalle de pedido con botón "Avanzar estado"), reutilizando los mismos validadores y reglas de negocio que la API.
- **Consulta de choferes:** `GET /api/choferes` devuelve el seed de choferes para usarlo como referencia (dropdown) al armar un pedido.

6. Módulos del proyecto

Módulo de Clientes

Listar, crear, editar y eliminar clientes.

Método	Ruta	Descripción
GET	<code>/api/clientes</code>	Lista todos los clientes
GET	<code>/api/clientes/:id</code>	Devuelve un cliente por id (<code>404</code> si no existe)
POST	<code>/api/clientes</code>	Crea un cliente (<code>201</code>); valida los 5 campos obligatorios
PUT	<code>/api/clientes/:id</code>	Edición parcial de un cliente
DELETE	<code>/api/clientes/:id</code>	Elimina un cliente

Vistas web en `/clientes`: listado, formulario de alta (`/clientes/nuevo`), formulario de edición (`/clientes/:id/editar`) y baja.

Archivos: `models/Cliente.js`, `controllers/clientesController.js`, `routes/api/clientesRoutes.js`, `routes/web/clientesRoutes.js`, `views/clientes/`.

Módulo de Pedidos

Listar, crear, editar y eliminar pedidos; ver y cambiar su estado.

Método	Ruta	Descripción
GET	/api/pedidos	Lista todos los pedidos
GET	/api/pedidos/:id	Devuelve un pedido por id (404 si no existe)
POST	/api/pedidos	Crea un pedido en estado pendiente (201); valida ítems y existencia de cliente y chofer
PUT	/api/pedidos/:id	Edición parcial (cliente, chofer, ítems, fecha); el estado no se cambia por acá
PATCH	/api/pedidos/:id/estado	Avanza al estado siguiente; rechaza saltos y retrocesos con 400
DELETE	/api/pedidos/:id	Elimina un pedido
GET	/api/choferes	Seed de choferes de solo lectura (referencia para asignar pedidos)

Vistas web en /pedidos : listado con nombre de cliente y chofer, detalle con botón "Avanzar estado" (/pedidos/:id), formulario de alta con select de cliente y chofer (/pedidos/nuevo), edición y baja.

Archivos: models/Pedido.js , controllers/pedidosController.js , services/PedidoService.js , routes/api/pedidosRoutes.js , routes/web/pedidosRoutes.js , views/pedidos/ .

Componentes transversales

- repositories/JsonRepository.js : clase genérica con getAll , getById , create , update y delete sobre un archivo JSON. Genera el id internamente (max(id) + 1).
- middlewares/logger.js , middlewares/validate.js , middlewares/errorHandler.js .

Próximos Módulos

- **Choferes (CRUD completo):** hoy es un seed fijo de 4 registros; en la segunda entrega pasa a ser un módulo con alta, edición y baja, y con asignación de zona.
- **Migración a MongoDB:** reemplazar JsonRepository por modelos de Mongoose manteniendo las mismas rutas y controladores.
- Posibles extensiones del caso (fuera del alcance actual): autenticación de usuarios, historial de cambios de estado por pedido.

7. Persistencia de Datos

Los datos se guardan en archivos JSON dentro de la carpeta /data/ del proyecto. Cada módulo instancia un JsonRepository con el nombre de su archivo y lee/escribe con fs.readFileSync / fs.writeFileSync .

Archivo	Registros en el seed	Campos
data/clientes.json	5	id, nombre, direccion, zona, contacto, horarioEntrega
data/pedidos.json	3	id, clienteId, choferId, items[{ descripcion, cantidad }], estado, fechaHoraProgramada
data/choferes.json	4	id, nombre, apellido, vehiculo, zona (solo lectura)

El repositorio se entrega con el seed cargado, de modo que al clonar y ejecutar `npm install && npm run dev` la aplicación ya muestra datos desde el primer request.

Capturas de Postman

Colección: `docs/api-collection.postman.json`. Capturas en `docs/evidencia/`.

Módulo Clientes

Postman interface showing a GET request to `{{baseUrl}} /api/clientes` with a successful response (200 OK). The response body is a JSON array of client data, displayed in a table format.

	id	nombre	direccion	zona	contacto	horarioEntrega
0	1	Restaurante La Esquina	Av. Corrientes 1234, CABA	Centro	Marcela Ruiz - 11 5555 1234	08:00 a 10:00
1	2	Comedor Escolar San Martín	Calle 12 N° 456, Quilmes	Sur	Jorge Peralta - 11 5555 5678	06:00 a 08:00
2	3	Cocina Industrial NorteAlimentos	Ruta 9 Km 42, Escobar	Norte	Ana Bianchi - 11 5555 9012	05:00 a 07:00

GET /api/clientes

HTTP FreshRoute B2B > Clientes > GET cliente por id Save Share

GET Send `{{baseUrl}} /api/clientes/1`

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 11 ms • 410 B • Save Response ...

`{}` JSON Preview Visualize

```
1 {
2   "id": 1,
3   "nombre": "Restaurante La Esquina",
4   "direccion": "Av. Corrientes 1234, CABA",
5   "zona": "Centro",
6   "contacto": "Marcela Ruiz - 11 5555 1234",
7   "horarioEntrega": "08:00 a 10:00"
8 }
```

GET /api/clientes/:id

FreshRoute B2B > Clientes > **POST crear cliente** Save Share

POST `{{baseUrl}} /api/clientes` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results **201 Created** • 21 ms • 437 B • Save Response ...

{} JSON Preview Visualize

```
1 {
2   "mensaje": "Cliente creado",
3   "cliente": {
4     "id": 4,
5     "nombre": "Bar Nuevo",
6     "direccion": "Calle 1 N° 100, Morón",
7     "zona": "Oeste",
8     "contacto": "Pedro Díaz - 11 0000 0000",
9     "horarioEntrega": "09:00 a 11:00"
10  }
11 }
```

POST /api/clientes

FreshRoute B2B > Clientes > **PUT actualizar cliente** Save Share

PUT `{{baseUrl}} /api/clientes/3` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 180 ms • 459 B • Save Response

JSON Preview Visualize

```

1  {
2    "mensaje": "Cliente actualizado",
3    "cliente": {
4      "id": 3,
5      "nombre": "Cocina Industrial NorteAlimentos",
6      "direccion": "Ruta 9 Km 42, Escobar",
7      "zona": "Centro",
8      "contacto": "Ana Bianchi - 11 5555 9012",
9      "horarioEntrega": "10:00 a 12:00"
10   }
11 }
```

PUT /api/clientes/id

FreshRoute B2B > Clientes > **DELETE eliminar cliente** Save Share

DELETE `{{baseUrl}} /api/clientes/2` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 4 ms • 268 B • Save Response

JSON Preview Visualize

```

1  {
2    "mensaje": "Cliente eliminado"
3  }
```

DELETE /api/clientes/id

FreshRoute B2B > Clientes > **GET cliente inexistente (404)** Save Share

GET `{{baseUrl}} /api/clientes/99` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 404 Not Found • 10 ms • 279 B • Save Response

Thinking...

JSON Preview Debug with AI

```

1 {
2   "mensaje": "Cliente no encontrado"
3 }
```

GET /api/clientes/:id inexistente → 404

FreshRoute B2B > Clientes > **POST crear cliente body vacío (400)** Save Share

POST `{{baseUrl}} /api/clientes` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

raw **JSON** Schema Beautify

```

1 {}
```

Body 400 Bad Request • 125 ms • 517 B • Save Response

Add required fields to body Apply Fix Decline

JSON Preview Debug with AI

```

1 {
2   "mensaje": "Datos de cliente inválidos",
3   "errores": [
4     "nombre es obligatorio y debe ser texto",
5     "direccion es obligatoria y debe ser texto",
6     "zona es obligatoria y debe ser texto",
7     "contacto es obligatorio y debe ser texto",
8     "horarioEntrega es obligatorio y debe ser texto"
9   ]
10 }
```

POST /api/clientes con body vacío → 400

Módulo Pedidos

FreshRoute B2B > Pedidos > GET todos los pedidos

SaveShare

GET{{baseUrl}}/api/pedidosSend

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

Body200 OK32 ms883 BSave Response

JSONPreviewVisualize

	id	clienteId	choferId	Items	estado	fechaHoraProgramada						
0	1	1	1	<table><thead><tr><th>descripcion</th><th>cantidad</th></tr></thead><tbody><tr><td>Cajón de verduras frescas</td><td>10</td></tr><tr><td>Bandeja de carnes refrigeradas</td><td>5</td></tr></tbody></table>	descripcion	cantidad	Cajón de verduras frescas	10	Bandeja de carnes refrigeradas	5	pendiente	2026-09-16T08:00:00
	descripcion	cantidad										
	Cajón de verduras frescas	10										
Bandeja de carnes refrigeradas	5											
1	2	2	2	<table><thead><tr><th>descripcion</th><th>cantidad</th></tr></thead><tbody><tr><td>Leche entera 1L</td><td>40</td></tr><tr><td>Pan lactal</td><td>20</td></tr></tbody></table>	descripcion	cantidad	Leche entera 1L	40	Pan lactal	20	asignado	2026-09-16T06:30:00
	descripcion	cantidad										
	Leche entera 1L	40										
Pan lactal	20											
2	3	3	3	<table><thead><tr><th>descripcion</th><th>cantidad</th></tr></thead><tbody><tr><td>Pollo congelado</td><td>15</td></tr><tr><td>Papas prefritas</td><td>25</td></tr></tbody></table>	descripcion	cantidad	Pollo congelado	15	Papas prefritas	25	en tránsito	2026-09-16T05:15:00
	descripcion	cantidad										
	Pollo congelado	15										
Papas prefritas	25											

GET /api/pedidos

FreshRoute B2B > Pedidos > GET pedido por id

SaveShare

GET{{baseUrl}}/api/pedidos/1Send

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

Body200 OK5 ms469 BSave Response

JSONPreviewVisualize

```
1 {
2   "id": 1,
3   "clienteId": 1,
4   "choferId": 1,
5   "items": [
6     {
7       "descripcion": "Cajón de verduras frescas",
8       "cantidad": 10
9     },
10    {
11      "descripcion": "Bandeja de carnes refrigeradas",
12      "cantidad": 5
13    }
14  ],
15  "estado": "pendiente",
16  "fechaHoraProgramada": "2026-09-16T08:00:00"
17 }
```

GET /api/pedidos/:id

FreshRoute B2B > Pedidos > **POST crear pedido** Save Share

POST `{{baseUrl}} /api/pedidos` Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "clienteId": 4,
3   "choferId": 4,
4   "items": [
5     { "descripcion": "Cajón de frutas", "cantidad": 8 }
6   ],
7   "fechaHoraProgramada": "2026-09-17T07:00:00"
8 }
```

Body Cookies Headers 7 Test Results 201 Created • 29 ms • 438 B • Save Response

`{}` JSON Preview Visualize

```
1 {
2   "mensaje": "Pedido creado",
3   "pedido": {
4     "id": 4,
5     "clienteId": 4,
6     "choferId": 4,
7     "items": [
8       {
9         "descripcion": "Cajón de frutas",
10        "cantidad": 8
11      }
12    ],
13    "estado": "pendiente",
14    "fechaHoraProgramada": "2026-09-17T07:00:00"
15  }
16 }
```

POST /api/pedidos

HTTP FreshRoute B2B > Pedidos > PUT actualizar pedido Save Share

PUT {{baseUrl}} /api/pedidos/2 Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "fechaHoraProgramada": "2026-09-16T07:00:00"
3 }
```

Body Cookies Headers 7 Test Results 200 OK • 29 ms • 480 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "mensaje": "Pedido actualizado",
3   "pedido": {
4     "id": 2,
5     "clienteId": 2,
6     "choferId": 2,
7     "items": [
8       {
9         "descripcion": "Leche entera 1L",
10        "cantidad": 40
11      },
12      {
13        "descripcion": "Pan lactal",
14        "cantidad": 20
15      }
16    ],
17    "estado": "asignado",
18    "fechaHoraProgramada": "2026-09-16T07:00:00"
19  }
```

PUT /api/pedidos/:id

FreshRoute B2B > Pedidos > **PATCH cambiar estado** Save Share

PATCH `{{baseUrl}} /api/pedidos/1/estado` Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "estado": "asignado"
3 }
```

Body Cookies Headers 7 Test Results 200 OK • 28 ms • 511 B • Save Response

{} JSON Preview Visualize

```
1 {
2   "mensaje": "Estado actualizado",
3   "pedido": {
4     "id": 1,
5     "clienteId": 1,
6     "choferId": 1,
7     "items": [
8       {
9         "descripcion": "Cajón de verduras frescas",
10        "cantidad": 10
11      },
12      {
13        "descripcion": "Bandeja de carnes refrigeradas",
14        "cantidad": 5
15      }
16    ],
17    "estado": "asignado",
18    "fechaHoraProgramada": "2026-09-16T08:00:00"
```

PATCH /api/pedidos/:id/estado

FreshRoute B2B > Pedidos > **PATCH cambiar estado inválido (400)** Save Share

PATCH `{{baseUrl}} /api/pedidos/2/estado` Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "estado": "entregado"
3 }
```

Body Cookies Headers 7 Test Results **400 Bad Request** • 28 ms • 309 B • Save Response ...

Investigate valid state transitions Diagnose Decline ...

{ } JSON Preview Debug with AI

```
1 {
2   "mensaje": "No se puede pasar de \"asignado\" a \"entregado\""
3 }
```

PATCH /api/pedidos/:id/estado transición inválida → 400

FreshRoute B2B > Pedidos > **DELETE eliminar pedido** Save Share

DELETE `{{baseUrl}} /api/pedidos/3` Send

Docs **Params** Authorization Headers 6 Body Scripts Settings Cookies

Body Cookies Headers 7 Test Results **200 OK** • 16 ms • 267 B • Save Response ...

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Pedido eliminado"
3 }
```

DELETE /api/pedidos/:id

FreshRoute B2B > Pedidos > **GET pedido inexistente (404)** Save Share

GET `{{baseUrl}} /api/pedidos/99` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body Cookies Headers 7 Test Results **404 Not Found** • 5 ms • 278 B • Save Response ...

Investigate 404 Not Found error Diagnose Decline ...

{ } JSON Preview Debug with AI

```
1 {
2   "mensaje": "Pedido no encontrado"
3 }
```

GET /api/pedidos/:id inexistente → 404

POST `{{baseUrl}} /api/pedidos` **Send**

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```

1 {
2   "clienteId": 1,
3   "choferId": 1,
4   "items": [],
5   "fechaHoraProgramada": "2026-09-17T07:00:00"
6 }

```

Body Cookies Headers 7 Test Results **400 Bad Request** • 5 ms • 366 B • e.g. Save Response ...

✦ Add items to request body ▾ **Apply Fix** **Decline** ...

{ } JSON ▾ Preview Debug with AI ▾

```

1 {
2   "mensaje": "Datos de pedido inválidos",
3   "errores": [
4     "items es obligatorio y debe ser un arreglo con al menos un item"
5   ]
6 }

```

POST /api/pedidos sin items → 400

Choferes (solo lectura)

GET `{{baseUrl}} /api/choferes` **Send**

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

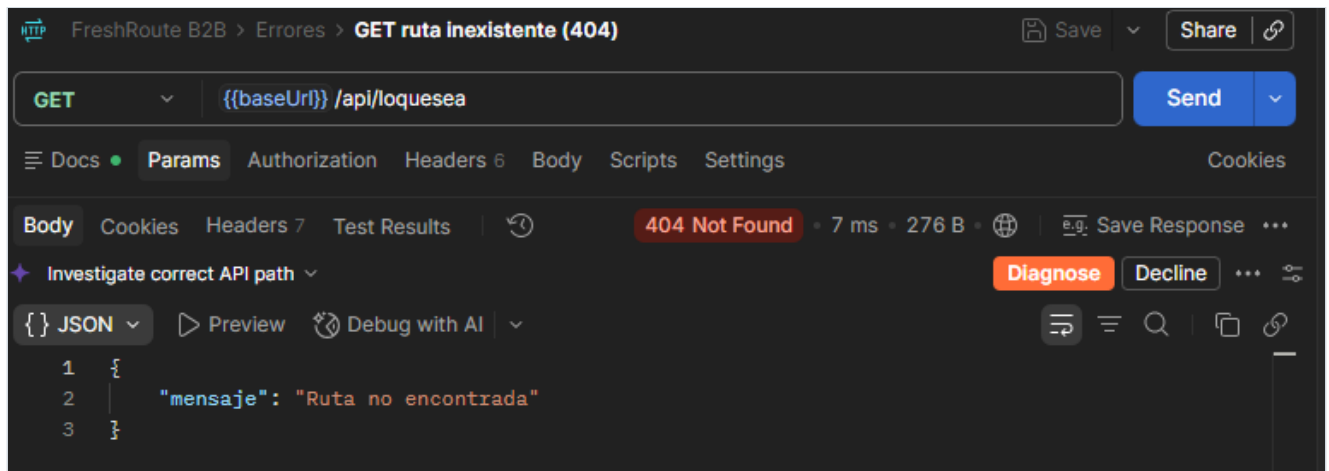
Body Cookies Headers 7 Test Results **200 OK** • 5 ms • 680 B • e.g. Save Response ...

{ } JSON Preview Visualize ▾

	Id	nombre	apellido	vehiculo	zona
0	1	Carlos	Gómez	Camioneta refrigerada - AB 123 CD	Norte
1	2	Lucía	Fernández	Camión refrigerado - AC 456 EF	Sur
2	3	Martín	Rodríguez	Camioneta refrigerada - AD 789 GH	Oeste
3	4	Sofía	López	Camión refrigerado - AE 012 IJ	Centro

GET /api/choferes

Manejo de errores



GET ruta inexistente → 404